INTERNSHIP REPORT

PFE - Final Year Project

DRUPAL DEVELOPMENT GUIDE

Version : Drupal 9/10

Full Name : Ayoub Barahal

Duration : 5 Days

Status : Done

"From site building mastery to custom module development."

Table des matières

1 Introduction **2**

2 Day 1 : Module Development Basics **2**
2.1 Step 4 : Testing Module Behavior . . . 2
2.1.1 1. Uninstall and Reinstall Module . . . 2
2.1.2 2. Lower core version requirement . . . 3
2.1.3 3. Add ‘workflows‘ as a Dependency . . . 3

3 Routes, Menus, Controllers and Services **3**
3.1 1. Creating a Menu Link . . . 3
3.2 2. Creating a Child Menu . . . 3
3.3 3. Retrieving a Query String in a Controller . . . 4
3.4 4. Guzzle and Logger . . . 4
3.4.1 Guzzle . . . 4
3.4.2 Logger . . . 4
3.5 5. Logging Messages in a Controller . . . 4
3.6 6. Drupal Core Services . . . 4
3.7 7. Injecting Services into a Custom Service . . . 5
3.8 8. Returning a Twig Template from a Controller . . . 5
3.9 9. Adding External JavaScript (Tailwind) . . . 5
3.10 10. Translation Keyword . . . 5
3.11 11. Making Strings Translatable in JavaScript . . . 5
3.12 12. Returning a JSON Response . . . 6

4 Day 3 : Hooks **6**
4.1 Metatag Module Hooks . . . 6
4.2 Modify a Form Hook . . . 6
4.3 Front Page Detection . . . 6

5 Day 4 : Plugins & Forms **7**
5.1 Meta Tag Viewport . . . 7
5.2 Add CSS Class to Menus . . . 7
5.3 Change Logo in Branding Block . . . 7
5.4 Drush PHP Config Dump . . . 7

6 Day 5 : Data Types and Entity API **8**
6.1 Why do we need a `schema.yml` file ? . . . 8
6.2 Loading a Node . . . 8
6.3 Getting Field Values . . . 8
6.4 Updating a Field and Saving the Node . . . 8
6.5 What is a Constraint ? . . . 8
6.6 What is a View Builder ? . . . 8
6.7 Entity Query and `accessCheck` . . . 9
6.8 Getting a Node Translation . . . 9

1 Introduction

This sprint focuses on Drupal module development, including hooks, services, forms, plugins, and entity queries. Instead of using the admin interface, we explore what happens behind the scenes at the code level.

Prerequisites

Before starting this sprint, you should have :

- Watched Create a Custom Module Course
- Watched Your First Drupal 8/9 Module
- Set up a GitHub repository to store your module work

Learning Outcomes

After this sprint, you will be able to :

- Create and configure custom Drupal modules
- Implement hooks and plugins
- Use Forms API
- Use and inject services
- Handle nodes, fields, and entity queries

2 Day 1 : Module Development Basics

2.1 Step 4 : Testing Module Behavior

2.1.1 1. Uninstall and Reinstall Module

Commands :

```
# Uninstall
./vendor/bin/drush pmu hello-world -y
./vendor/bin/drush cr

# Reinstall
./vendor/bin/drush en hello-world -y
./vendor/bin/drush cr
```

Results :

- **Uninstall :** Drupal removes all module configuration, stops hooks from firing, drops custom tables, and removes the module from the active list.
- **Reinstall :** Drupal re-runs `hook_install()`, recreates tables, re-imports config — effectively a fresh module install.
- If no `hook_install()` or schema exists, nothing major happens.

2.1.2 2. Lower core version requirement

Attempting to install a module requiring a higher core version fails. Example : unable to install module because it requires Drupal core >= 9.x.x.

2.1.3 3. Add ‘workflows‘ as a Dependency

Drush installs ‘workflows‘ first, then your module, ensuring all dependencies are met.

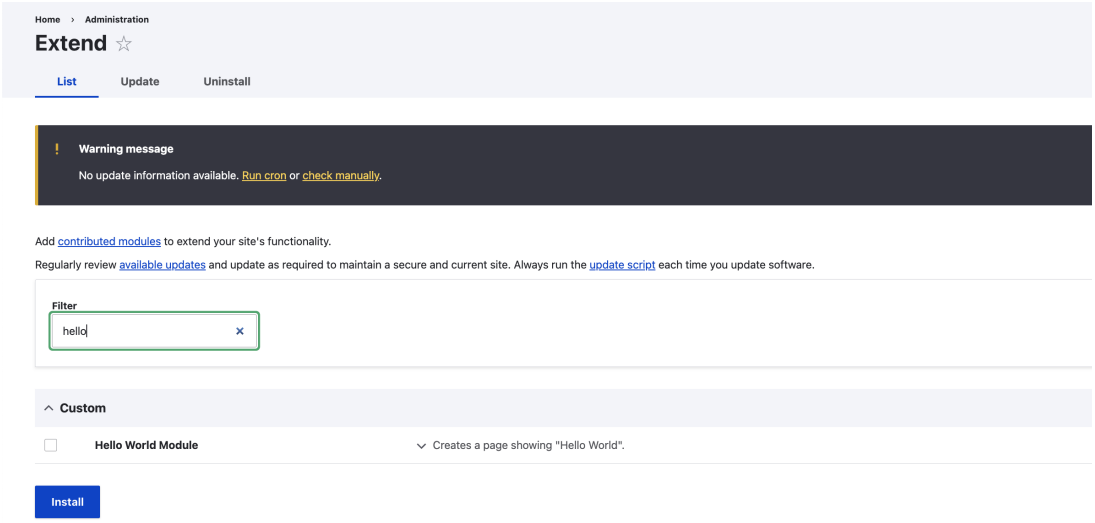

FIGURE 1 – hello world module

3 Routes, Menus, Controllers and Services

3.1 1. Creating a Menu Link

One way to create a menu link in Drupal is by defining it in the file :

`mymodule.links.menu.yml`

Example

```
mymodule.my_link:
  title: 'My link'
  route_name: mymodule.my_route
  parent: system.admin_content
  weight: 5
```

This configuration creates a new menu item inside the Drupal administration content menu.

3.2 2. Creating a Child Menu

To create a child menu (sub-menu), we use the `parent` property in the same YAML file.

Example

```
mymodule.child_link:
  title: 'Child link'
  route_name: mymodule.child_route
  parent: mymodule.parent_link
```

This will create a submenu under `parent_link`.

3.3 3. Retrieving a Query String in a Controller

A query string is different from a route parameter.

Example URL

```
/my-page?name=ayoub
```

Inside the controller, the query parameter can be retrieved using :

```
$name = \Drupal::request()->query->get('name');
```

Alternatively, this can also be done using the `request_stack` service via dependency injection.

3.4 4. Guzzle and Logger

3.4.1 Guzzle

Guzzle is a PHP HTTP client used to send HTTP requests to external APIs or web services. Drupal uses Guzzle internally to handle HTTP communications.

3.4.2 Logger

The Logger service allows developers to record messages such as debug information, warnings, or errors.

3.5 5. Logging Messages in a Controller

When using the Logger service, messages appear in the Drupal logging interface :

```
/admin/reports/dblog
```

This page displays all logged events in the Drupal database.

3.6 6. Drupal Core Services

All Drupal core services are defined in the following YAML file :

```
core/core.services.yml
```

This file contains the definitions of services used by Drupal's dependency injection container.

3.7 7. Injecting Services into a Custom Service

Services can be injected through the module service configuration file :

`mymodule.services.yml`

```
services:
  mymodule.my_service:
    class: Drupal\mymodule\MyService
    arguments: ['@logger.factory', '@path.alias_manager']
```

This example injects the Logger factory service and the Path Alias Manager service.

3.8 8. Returning a Twig Template from a Controller

A Twig template controls the visual rendering and layout of content in Drupal.

```
return [
  '#theme' => 'my_template',
  '#name' => 'ayoub',
];
```

The variable `name` can then be used inside the Twig template.

3.9 9. Adding External JavaScript (Tailwind)

To add external JavaScript such as Tailwind, a library must first be declared in :

`mymodule.libraries.yml`

```
tailwind:
  js:
    https://unpkg.com/@tailwindcss/browser@4: { type: external }
```

This library can then be attached to pages or render arrays.

3.10 10. Translation Keyword

When using the translation function :

```
$this->t('Hello, @name!', ['@name' => $username])
```

The searchable keyword in the translation interface is :

`Hello`

3.11 11. Making Strings Translatable in JavaScript

In Drupal JavaScript files, the translation function is :

```
Drupal.t('Hello world');
```

This ensures that the string can be translated through Drupal's localization system.

3.12 12. Returning a JSON Response

Instead of returning a page or Twig template, a controller can return a JSON response.

```
use Symfony\Component\HttpFoundation\JsonResponse;

return new JsonResponse([
  'name' => 'ayoub',
  'status' => 'ok',
]);
```

This is commonly used for APIs or AJAX responses.

4 Day 3 : Hooks

4.1 Metatag Module Hooks

Located in `web/modules/contrib/metatag/`, examples :

- info
- info_alter
- defaults_alter
- attachments_alter

4.2 Modify a Form Hook

```
function mymodule_form_alter(&$form,
    \Drupal\Core\Form\FormStateInterface $form_state, $form_id) {
  if ($form_id == 'user_login_form') {
    $form['my_custom_textfield'] = [
      '#type' => 'textfield',
      '#title' => t('My Custom Field'),
      '#required' => FALSE,
      '#description' => t('This is a custom text field added to the
          login form.'),
    ];
  }
}
```

4.3 Front Page Detection

```
$variables['is_front'] =
    \Drupal::service('path.matcher')->isFrontPage();
```

Twig usage :

```
{{ dump(is_front) }}
{% if is_front %}
  <h1>Welcome to the front page!</h1>
```

```
{% endif %}
```

5 Day 4 : Plugins & Forms

5.1 Meta Tag Viewport

```
function mon_theme_page_attachments_alter(array &$attachments) {
  $attachments['#attached']['html_head'][] = [
    [
      '#tag' => 'meta',
      '#attributes' => [
        'name' => 'viewport',
        'content' => 'width=device-width, initial-scale=1',
      ],
    ],
    'custom_viewport',
  ];
}
```

5.2 Add CSS Class to Menus

```
function mon_theme_preprocess_menu(&$variables) {
  foreach ($variables['items'] as &$item) {
    $item['attributes']['class'][] = 'my-custom-class';
  }
}
```

5.3 Change Logo in Branding Block

Twig example :

```
<a href="{{ front_page }}" class="site-logo">
  <img src="https://static.cdnlogo.com/logos/d/88/drupal-wordmark.svg"
     alt="Home" />
</a>
```

5.4 Drush PHP Config Dump

```
drush php:eval
   "print_r(\Drupal::config('eu_cookie_compliance.settings')->get());"
```

Configuration file location :

`module`$_{name/config/install/eu_cookie_compliance.settings.yml}$

6 Day 5: Data Types and Entity API

6.1 Why do we need a schema.yml file?

The schema file describes the structure of configuration objects.

It allows Drupal to:

- Validate configuration values
- Provide proper translation support
- Define the data types of configuration properties

6.2 Loading a Node

Nodes can be loaded using the Node entity class.

```
use Drupal\node\Entity\Node;

$node = Node::load(1);
```

6.3 Getting Field Values

Fields can be accessed through the entity API.

```
$title = $node->getTitle();
$body = $node->get('body')->value;
```

6.4 Updating a Field and Saving the Node

```
$node->set('title', 'Updated title');
$node->save();
```

6.5 What is a Constraint?

A constraint is a validation rule applied to entity fields.

Examples include:

- Required fields
- Maximum length
- Unique values

6.6 What is a View Builder?

View builders are responsible for rendering entities.

Drupal provides several display modes such as:

- Full content
- Teaser

- Custom display modes

These modes define how entity fields are displayed.

6.7 Entity Query and accessCheck

In entity queries, accessCheck(TRUE) ensures that only entities the user has permission to view are returned.

```
$query = \Drupal::entityQuery('node')
  ->accessCheck(TRUE);
```

6.8 Getting a Node Translation

Drupal entities support multilingual content.

```
$translated_node = $node->getTranslation('fr');
```

Conclusion

This training provided a practical introduction to Drupal development by exploring how the platform works beyond the administrative interface. Throughout the different sections, we learned how to create custom modules, work with routes, controllers, services, and dependency injection, and use hooks to extend Drupal's functionality.

We also practiced using the Form API, plugins, and entity systems to manipulate content, build forms, and query data programmatically. These exercises helped develop a better understanding of Drupal's architecture and coding standards.

Overall, this guide helped build a solid foundation in Drupal development and provided the essential skills needed to create custom features and extend Drupal websites in a structured and maintainable way.